StockScope - Interactive Stock & ETF Explorer

28 Sep, 2025

Weekly Team Sync - 1

Agenda

01 Momentum & Introductions

02 Review project vision + goals

03 Assign/confirm roles

04 Agree on tools & workflow

05 Next steps + Week 1 deliverables

01

Introductions

Project Vision

Web-based dashboard that helps investors make informed decisions by integrating fundamental, technical, and sentiment data into one interactive platform.

01

AI-generated investment summaries (NLP-driven sentiment + data insights)

02

Real-time stock & ETF charts with overlays: MACD, RSI, Bollinger Bands

03

News/Sentiment Analysis from financial sources/social media

04

Data integration via Alpha Vantage & Yahoo Finance APIs

Project Vision

Industry terms

01

NLP-driven sentiment- detected from text using natural language processing to classify opinions.

02

Stocks & ETF - Tradable financial assets where stocks represent ownership in a company, and ETFs are diversified funds that trade like stocks.

03

MACD - momentum indicator showing the relationship between two moving averages of a security's price, used to identify trends and potential buy/sell signals

04

RSI - momentum oscillator that measures the speed and change of price movements to identify overbought or oversold conditions, typically on a scale of 0–100

05

Bollinger Bands - volatility indicator consisting of a moving average and upper and lower bands, which expand or contract based on price volatility, helping identify overbought or oversold conditions

Goals & Deliverables

Frontend Team (3 developers)

- Interactive Stock & ETF Dashboard
- News & Sentiment Visualization Module

Backend Team (3 developers)

- Data Pipeline & API Integration
- Flask API for Frontend Consumption

AI/NLP Team (3 developers)

- Sentiment Analysis Engine
- AI-generated Investment Summaries

Roles

Front-End Development

Key Skills:

- React.js: Component-based UI development
- Plotly / D3.js: Interactive charting and data visualization
- HTML/CSS & JavaScript: Responsive, clean UI design
- State management: Redux or Context API
- Version control: Git/GitHub workflows

Interests / Motivations:

- Creating interactive dashboards and user-friendly interfaces
- Working with real-time data visualizations
- Experimenting with UI/UX design and animations
- Enjoy problem-solving for data-heavy frontends

Back-End Development

Key Skills:

- Python & Flask: API development and server-side logic
- Database management: SQL/NoSQL (if storing historical data)
- API integration: REST APIs, JSON handling
- Data processing: Pandas, NumPy, or similar libraries
- Deployment basics: Hosting APIs, Docker, cloud platforms

Interests / Motivations:

- Building robust data pipelines
- Working with financial data and APIs
- Optimizing backend performance and scalability
- Creating infrastructure that powers interactive apps

Roles

AI/NLP Development

Key Skills:

- Natural Language Processing (NLP): Sentiment analysis, text summarization
- Python ML libraries: spaCy, NLTK, Transformers, or Hugging Face
- Data analysis: Pandas, NumPy for preprocessing
- API integration: Feeding AI outputs to frontend/backends
- Optional: Fine-tuning or training small models

Interests / Motivations:

- Analyzing news and social media to extract insights
- Generating actionable summaries from raw text
- Experimenting with AI-driven financial insights
- Working at the intersection of finance, data, and AI

Tools & Workflows

Tech Stack

Data Flow:

1. APIs → Backend (Flask) → Compute indicators & AI analysis
2. Backend → Frontend (React + Plotly/D3) → Interactive dashboard

01 Frontend:

- React.js – Dynamic, component-based UI
- Plotly / D3.js – Interactive stock/ETF charts & visualizations

02 Backend:

- Flask (Python) – API handling, data processing, technical indicators
- Data Integration – Fetch & clean data from Alpha Vantage & Yahoo Finance

03 AI / NLP:

- Sentiment Analysis – News & social media sentiment scoring
- Summarization – AI-generated actionable investment summaries

Tools & Workflows

Workflow

Slack / Discord – Daily communication, quick questions, announcements

Google Drive – Store specs, meeting slides, API references, demo videos and any other references/ project documentation

GitHub – Version control, code reviews, weekly documentation README and pull requests

Trello / Jira / Notion – Task assignment, deadlines, workflow tracking (optional)

01 Weekly Team Sync - Weekend sessions (~60–90 min) for in-depth progress review, planning, and milestone alignment

02 Weekly Check-in - Midweek (~30 min, Wed/Thu) to quickly assess progress, address blockers, and make minor adjustments

03 Mid-semester review - Demo MVP, assess milestones, adjust scope, and ensure project is on track.

Note on Git

Pull before you push – Always git pull origin develop before starting work to avoid conflicts

Branching:

- Each task or feature gets its own branch
- Example: Frontend dev working on charts → feature/frontend-charts

Commit often & clearly:

- Use meaningful messages: feat(frontend): add RSI overlay to stock chart
- Small, incremental commits are easier to review and merge

Push & Create Pull Requests:

- Push feature branch to GitHub
- Open a Pull Request (PR) into develop
- Team members review, escalate pull request to PM then merge after approval

Tools & Workflows

Feature Development on Your NLP Branch

Create a branch from the latest nlp branch:
git checkout nlp
git pull origin nlp
git checkout -b nlp-myfeature

Complete your code and review formatting (lint, type checks, etc.)

Test locally to ensure functionality works

Commit with a descriptive message:
git add .
git commit -m "Add XYZ functionality: brief description"

Pull Request to Team Branch (nlp)

Push your branch:
git push origin nlp-myfeature

Open PR targeting the nlp branch (not main)

Assign team member for review

Add a message:
Functionality: XYZ feature implemented
Notes: [any relevant info, caveats, or pending tests]

Check for conflicts.

If there are conflicts between nlp-myfeature and nlp:
git fetch origin
git checkout nlp
git pull origin nlp
git checkout nlp-myfeature
git merge nlp
Resolve conflicts, then:
git add .
git commit -m "Resolve merge conflicts"
git push origin nlp-myfeature

Once reviewed and approved, merge into nlp.

Pull Request to main Branch

After nlp branch is updated, open PR from nlp → main

Assign Project Manager for review

Include message:
Functionality: XYZ feature merged from NLP team
Notes: [any additional info, impact, or documentation references]

PM reviews, approves, and merges

Update documentation if needed

Next Steps

1. Finalize team roles and responsibilities
2. Create GitHub repo and familiarize with version control
3. Establish communication channels
4. Prepare initial project documentation conventions
5. Complete Week 1 tasks

Weekly Team Sync 1 - 9/21 - Summary

Project Vision:
Web-based dashboard for investors integrating fundamental, technical, and sentiment data-helps users make informed decisions

Core Features (MVP):
AI-generated investment summaries (NLP)
Real-time stock/ETF charts (MACD, RSI, Bollinger Bands)
News & sentiment analysis
Data from Alpha Vantage & Yahoo Finance

Tech Stack:
Frontend: React.js + Plotly/D3.js
Backend: Flask (API & data pipeline)
AI/NLP: Sentiment analysis + summaries
Data Flow: APIs → Backend → AI/NLP → Frontend

Team & Deliverables:
Frontend: Interactive charts, sentiment visualization
Backend: API integration, technical indicators
AI/NLP: Sentiment engine, investment summaries

Workflow & Communication:
Slack/Discord: Daily updates
Trello/Jira/Notion: Task tracking (?)
GitHub: Version control
Google Drive/Notion: Documentation

Regular Meetings:
Weekly Sync (Weekend): Deep-dive progress & planning
Weekly Check-in (Wed/Thu): Midweek progress review

Week 1 Deliverables:
React + Flask skeletons
Mock sentiment analysis module
GitHub repo & task board setup
Communication channels established
Project vision, goals, and roles documented